Rapport de Projet
Programmation Internet
Système de Partage de Fichiers P2P

Guillaume Xue : 22101031
David Yu : 22110478

24 janvier 2026

Table des matières

1 Introduction **2**
1.1 Objectifs du projet 2
1.2 Architecture générale 2

2 Implementation minimum **3**
2.1 Enregistrement serveur central 3
2.1.1 Enregistrement de la clé publique 3
2.1.2 Enregistrement de l'adresse IP et du port 3
2.1.3 Maintien de la connexion 4
2.2 Partage de fichiers et répertoires 5
2.2.1 Structure de l'arbre de Merkle 5
2.3 Téléchargement de fichiers 6
2.3.1 Architecture générale 6
2.3.2 Fenêtre glissante adaptative 6
2.3.3 Gestion des timeouts et retransmissions 6
2.3.4 Téléchargement récursif 6

3 Fonctionnalités avancées **8**
3.1 Interface utilisateur interactive 8
3.2 Gestion multi-protocoles IPv4/IPv6 8
3.2.1 Dual-Stack avec fallback automatique 8
3.2.2 Double enregistrement Dual-Stack 9
3.2.3 Gestion des adresses multiples par pair 9

Chapitre 1

Introduction

Ce projet crée un système de partage de fichiers peer-to-peer (P2P) avec un serveur central. Le serveur gère la découverte des pairs et l'échange des clés. Tout communique en UDP et on utilise un arbre de Merkle pour vérifier que les données ne sont pas corrompues.

1.1 Objectifs du projet

Le client doit pouvoir :
— S'inscrire auprès du serveur et rester connecté
— Partager ses fichiers avec les autres
— Télécharger des fichiers depuis d'autres utilisateurs
— Fonctionner même derrière un NAT
— Gérer les téléchargements efficacement avec une fenêtre glissante

1.2 Architecture générale

On l'a écrit en Go avec plusieurs parties :
— **Transport** : Gère la communication en UDP
— **Merkle** : Crée les arbres Merkle pour les fichiers
— **Crypto** : Gère les clés et les signatures
— **Peer** : Tient à jour la liste des autres utilisateurs
— **Protocol** : Définit les types de messages

Chapitre 2

Implementation minimum

2.1 Enregistrement serveur central

2.1.1 Enregistrement de la clé publique

D'abord, on envoie notre clé publique au serveur central. Le serveur la stocke pour que les autres puissent la récupérer et vérifier nos signatures.

2.1.2 Enregistrement de l'adresse IP et du port

Pour enregistrer notre adresse IP, on utilise `TryConnectWithFallback`. On essaie d'abord de se connecter en IPv6, et si ça ne marche pas, on bascule sur IPv4.

Connexion IPv4/IPv6

La fonction `TryConnectWithFallback` fonctionne en deux étapes :

Étape 1 : Ouvrir la connexion On essaie d'abord d'ouvrir un socket IPv6 :

```
addrV6, _ := net.ResolveUDPAddr("udp", "[::]:0")
conn, err = net.ListenUDP("udp", addrV6)
```

Si ça ne marche pas, on bascule sur IPv4 :

```
addrV4, _ := net.ResolveUDPAddr("udp4", "0.0.0.0:0")
conn, err = net.ListenUDP("udp4", addrV4)
```

On met le port à 0 pour que le système choisisse un port libre automatiquement.

Étape 2 : Se connecter au serveur Après, on détermine quelle adresse du serveur utiliser. On préfère IPv6, sinon on essaie IPv4 :

```
if isDualStack {
    targets = append(targets, protocol.GetServerUDPv6())
}
targets = append(targets, protocol.GetServerUDPv4())
```

Pour chaque adresse du serveur, on :

1. Envoie un `Hello` signé avec notre clé privée
2. On attend une réponse (max 2 secondes)
3. On vérifie que c'est bien la bonne réponse en regardant l'ID

Attendre la réponse

`waitResponse` attend la réponse du serveur :

```
deadline := time.Now().Add(2 * time.Second)
conn.SetReadDeadline(deadline)
```

Elle vérifie que :
— L'ID correspond à celui qu'on a envoyé
— Le type de message est correct

Double connexion

Quand on est connecté en IPv6, on envoie aussi un ping sur IPv4 :

```
if isDualStack && targetStr == protocol.GetServerUDPv6() {
    if otherAddr, err := net.ResolveUDPAddr("udp",
                          protocol.GetServerUDPv4()); err == nil {
        SendHello(conn, otherAddr, name, privKey)
    }
}
```

Ça permet au serveur de nous enregistrer sur les deux protocoles.

2.1.3 Maintien de la connexion

Une fois connecté, on doit rester en contact avec le serveur et les autres utilisateurs. On utilise des pings réguliers pour ça.

Mécanisme de keep-alive

La fonction `KeepAlive` s'exécute en arrière-plan et envoie régulièrement des pings. Elle utilise un ticker pour déclencher les pings :

```
func (s *Server) KeepAlive(serverAddr *net.UDPAddr,
                           interval time.Duration, ctx context.Context) {
    ticker := time.NewTicker(interval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-s.shutdown:
            return
        case <-ticker.C:
            // Logique de ping...
        }
    }
}
```

Stratégie de ping

À chaque cycle, on ping tous les autres utilisateurs connectés. Pour chaque utilisateur, on envoie un ping sur toutes ses adresses :

```
connectedPeers := s.Manager.List()
if len(connectedPeers) > 0 {
    for _, peerName := range connectedPeers {
```

```go
        if peerInfo, ok := s.Manager.Get(peerName); ok {
            for _, addrInfo := range peerInfo.Addrs {
                SendPing(s.Conn, addrInfo.Addr)
                time.Sleep(1 * time.Millisecond)
            }
        }
    }
}
```

On attend 1 milliseconde entre chaque ping pour éviter de surcharger le réseau.

Après avoir envoyé tous les pings, on nettoie la liste :

```go
s.Manager.CleanExpired()
```

Ça supprime les utilisateurs qui n'ont pas répondu depuis longtemps. Cette approche garantit que :

- Les autres savent qu'on est actif
- Les vieux utilisateurs sont enlevés de la liste
- Les NAT gardent les ports ouverts

Traiter les pings reçus

Quand on reçoit un ping d'un autre utilisateur, on répond immédiatement :

```go
case protocol.Ping:
    SendOk(s.Conn, addr, pkt.Header.ID)
```

À chaque message reçu (pas juste les pings), on met à jour le timestamp de l'utilisateur pour savoir qu'il est actif. Ça permet de détecter rapidement si quelqu'un n'est plus en ligne.

2.2 Partage de fichiers et répertoires

2.2.1 Structure de l'arbre de Merkle

On utilise un arbre de Merkle pour représenter les fichiers et répertoires. Chaque nœud a un hash SHA-256 unique, ce qui garantit que les données ne sont pas corrompues. Au démarrage, les fichiers locaux sont convertis en arbres Merkle.

Types de nœuds

Il y a quatre types de nœuds :

- **Type 0 (Chunk)** : Données brutes d'un fichier (max 1024 octets)
- **Type 1 (Directory)** : Répertoire simple (max 16 entrées)
- **Type 2 (Big)** : Fichier fragmenté en plusieurs chunks (max 32 enfants)
- **Type 3 (BigDirectory)** : Répertoire fragmenté (max 32 sous-répertoires)

Stockage des fichiers

Le module `merkle` gère le stockage des données avec une structure thread-safe :

```go
type Store struct {
    data map[[32]byte][]byte
    mu   sync.RWMutex
}

func (s *Store) Add(datum []byte) [32]byte { ... }
func (s *Store) Get(hash [32]byte) ([]byte, bool) { ... }
```

On a deux stores :
— `MerkleStore` : Fichiers qu'on partage
— `Downloads` : Fichiers qu'on a téléchargés depuis les autres

2.3 Téléchargement de fichiers

2.3.1 Architecture générale

Le téléchargement est géré par deux composants :
— **Downloader** : Télécharge en mémoire pour explorer les fichiers
— **DiskDownloader** : Télécharge et sauvegarde sur le disque
Un dispatcher centralisé gère les paquets reçus et peut notifier plusieurs downloader en même temps.

2.3.2 Fenêtre glissante adaptative

La fenêtre glissante contrôle le nombre de requêtes qu'on envoie en même temps. Elle s'ajuste selon la vitesse du réseau :

```go
type Downloader struct {
    windowSize    int
    maxWindowSize int
    minWindowSize int
    timeout       time.Duration

    pending map[[32]byte]time.Time
    retries map[[32]byte]int
}
```

Le downloader a trois workers qui tournent en parallèle :

1. **senderLoop** : Envoie les demandes en respectant la taille de la fenêtre
2. **responseLoop** : Traite les réponses et ajuste la fenêtre
3. **monitorLoop** : Vérifie les timeouts et renvoie ce qui n'a pas répondu

2.3.3 Gestion des timeouts et retransmissions

Le `monitorLoop` vérifie les demandes qui n'ont pas eu de réponse :
— Timeout : On renvoie la demande jusqu'à un max de tentatives
— Ça échoue trop : On réduit la fenêtre
— Abandon : On supprime si ça marche pas
La fenêtre s'ajuste :
— Si c'est rapide : On l'augmente
— Si c'est lent : On la réduit
— Limites : Entre le min et le max configurés

2.3.4 Téléchargement récursif

On télécharge tout l'arbre Merkle en descendant récursivement :

1. Demander le hash racine
2. Recevoir le fichier racine
3. Voir quel type c'est et extraire les hashes enfants
4. Ajouter les enfants à la file d'attente

5. Recommencer jusqu'à avoir tout

Ça permet de télécharger juste des fichiers ou des arborescences complètes selon ce qu'on veut.

Chapitre 3

Fonctionnalités avancées

3.1 Interface utilisateur interactive

L'appli propose un menu interactif pour :
- Voir les autres utilisateurs du réseau
- Se connecter à un utilisateur
- Explorer les fichiers d'un autre
- Télécharger des fichiers manuellement
- Voir les connexions actives
- Voir ses propres fichiers
- Activer le mode debug pour voir les paquets

Le module `menu` implémente ce menu avec :
- **Run** : La boucle principale du menu
- **listDirectoryPeers** : Affiche la liste des autres utilisateurs
- **connectToPeer** : Se connecte à quelqu'un
- **explorePeer** : Parcourt les fichiers d'un autre utilisateur
- **downloadManual** : Télécharge un fichier par son hash
- **showConnections** : Affiche qui est connecté
- **showLocalFiles** : Affiche ses propres fichiers

3.2 Gestion multi-protocoles IPv4/IPv6

On supporte les deux protocoles avec du fallback automatique.

3.2.1 Dual-Stack avec fallback automatique

`TryConnectWithFallback` se connecte en IPv6 d'abord, puis bascule sur IPv4.

Ouverture du socket local

On essaie d'ouvrir un socket IPv6 :

```
addrV6, _ := net.ResolveUDPAddr("udp", "[::]:0")
conn, err = net.ListenUDP("udp", addrV6)

if err == nil {
    isDualStack = true
} else {
    // Fallback IPv4
    addrV4, _ := net.ResolveUDPAddr("udp4", "0.0.0.0:0")
    conn, err = net.ListenUDP("udp4", addrV4)
```

```
}
```

Le port local est laissé à 0 pour que le système d'exploitation choisisse un port disponible, évitant ainsi les conflits « Address already in use ».

Se connecter au serveur

Après avoir ouvert le socket, on détermine quelle adresse du serveur utiliser :

```
if isDualStack {
    targets = append(targets, protocol.GetServerUDPv6())
}
targets = append(targets, protocol.GetServerUDPv4())
```

Pour chaque adresse cible, on :

1. Envoie un `Hello` signé avec notre clé
2. On attend une réponse (max 2 secondes)
3. On vérifie que c'est la bonne réponse

3.2.2 Double enregistrement Dual-Stack

Quand on est connecté en IPv6, on envoie aussi un ping sur IPv4 pour que le serveur nous enregistre sur les deux :

```
if isDualStack && targetStr == protocol.GetServerUDPv6() {
    if otherAddr, err := net.ResolveUDPAddr("udp",
                          protocol.GetServerUDPv4()); err == nil {
        SendHello(conn, otherAddr, name, privKey)
    }
}
```

3.2.3 Gestion des adresses multiples par pair

On stocke toutes les adresses (IPv4 et IPv6) de chaque utilisateur :

```
type PeerInfo struct {
    Name      string
    Addrs     []AddrInfo
    PublicKey *ecdsa.PublicKey
    LastSeen  time.Time
    IsRelay   bool
}

type AddrInfo struct {
    Addr     *net.UDPAddr
    LastSeen time.Time
}
```

Quand on reçoit une adresse d'un utilisateur, on :

— IPv4 connu → On remplace l'ancienne adresse IPv4
— IPv6 connu → On remplace l'ancienne adresse IPv6
— Nouveau protocole → On l'ajoute